In C#, properties are used to safely read, write, or control access to the data (fields) of a class.

```
using System;

class Student
{
    public string Name { get; set; }
    public int Age { get; set; }
}

class Program
{
    static void Main()
    {
        Student student = new Student();

        student.Name = "Rahul";
        student.Age = 20;

        Console.WriteLine("Student Name: " + student.Name);
        Console.WriteLine("Student Age: " + student.Age);
    }
}
```

.....

Access Modifiers in C#

Access modifiers in C# are keywords that control where a class, method, property, or variable can be accessed.

There are mainly 5 access modifiers:

Modifier	Access
public	Accessible from anywhere

private Accessible only inside the same class

protected Accessible inside the same class and derived class

internal Accessible within the same project/assembly

protected internal Accessible within the same assembly OR from a derived class

1. public

Can be accessed from anywhere.

```
using System;

class Student
{
   public string Name = "Rahul";
}

class Program
{
   static void Main()
   {
      Student s = new Student();

      Console.WriteLine(s.Name);
   }
}
```

2. private

Can be accessed only inside the same class.

```
using System;

class Student
{
   private int age = 20;
```

```
    public void Display()
    {
        Console.WriteLine(age);
    }
}

class Program
{
    static void Main()
    {
        Student s = new Student();

        s.Display();
    }
}
```

3. protected

Can be accessed inside the class and its child/derived class.

```
using System;

class Animal
{
    protected string name = "Dog";
}

class Dog : Animal
{
    public void Display()
    {
```

```
        Console.WriteLine(name);
    }
}

class Program
{
    static void Main()
    {
        Dog d = new Dog();

        d.Display();
    }
}
```

4. internal

Can be accessed from anywhere within the same project/assembly.

```
using System;

class Student
{
    internal string Name = "Rahul";
}

class Program
{
    static void Main()
    {
        Student s = new Student();

        Console.WriteLine(s.Name);
```

```
    }
}
```

5. protected internal

This combines protected and internal.

It can be accessed:

Within the same assembly/project, or

From a derived class, even in another assembly.

Example:

```
using System;

class Parent
{
    protected internal int number = 100;
}

class Child : Parent
{
    public void Display()
    {
        Console.WriteLine(number);
    }
}

class Program
{
    static void Main()
    {
        Child c = new Child();
```

```
        c.Display();
    }
}
```

.....

Static Members in C#

A static member belongs to the class itself, not to a particular object.

So, you can access a static member using the class name, without creating an object.

Example: Static Variable and Static Method

```
using System;

class Student
{
    // Static variable
    public static string CollegeName = "ABC College";

    // Non-static variable
    public string Name;

    // Static method
    public static void DisplayCollege()
    {
        Console.WriteLine("College: " + CollegeName);
    }
}

class Program
{
    static void Main()
    {
```

```
        // Access static variable using class name
        Console.WriteLine(Student.CollegeName);

        // Call static method using class name
        Student.DisplayCollege();

        // Create object for non-static member
        Student s = new Student();
        s.Name = "Rahul";

        Console.WriteLine("Student Name: " + s.Name);
    }
}
```

.....

Types of Inheritance in C#

Inheritance allows one class to acquire the properties and methods of another class.

C# mainly supports these types of inheritance:

Single Inheritance

Multilevel Inheritance

Hierarchical Inheritance

Multiple Inheritance through Interfaces

1. Single Inheritance

In single inheritance, one child class inherits from one parent class.

Animal

↓

Dog

Example

```
using System;

class Animal
{
   public void Eat()
   {
      Console.WriteLine("Animal is eating");
   }
}

class Dog : Animal
{
   public void Bark()
   {
      Console.WriteLine("Dog is barking");
   }
}

class Program
{
   static void Main()
   {
      Dog d = new Dog();

      d.Eat();
      d.Bark();
   }
}
```

Output

```
Animal is eating
```

Dog is barking

2. Multilevel Inheritance

In multilevel inheritance, a class inherits from another derived class.

Animal

↓

Mammal

↓

Dog

Example

```
using System;

class Animal
{
   public void Eat()
   {
      Console.WriteLine("Animal is eating");
   }
}

class Mammal : Animal
{
   public void Walk()
   {
      Console.WriteLine("Mammal is walking");
   }
}

class Dog : Mammal
{
```

```
    public void Bark()
    {
        Console.WriteLine("Dog is barking");
    }
}

class Program
{
    static void Main()
    {
        Dog d = new Dog();

        d.Eat();
        d.Walk();
        d.Bark();
    }
}
```

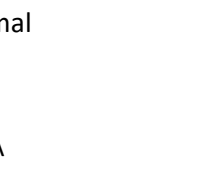
Output

```
Animal is eating
Mammal is walking
Dog is barking
```

Here, Dog gets members from both Mammal and Animal.

3. Hierarchical Inheritance

In hierarchical inheritance, multiple child classes inherit from the same parent class.



Dog Cat

Example

```
using System;

class Animal
{
   public void Eat()
   {
      Console.WriteLine("Animal is eating");
   }
}

class Dog : Animal
{
   public void Bark()
   {
      Console.WriteLine("Dog is barking");
   }
}

class Cat : Animal
{
   public void Meow()
   {
      Console.WriteLine("Cat is meowing");
   }
}

class Program
{
   static void Main()
```

```
    {
        Dog d = new Dog();

        d.Eat();
        d.Bark();

        Cat c = new Cat();

        c.Eat();
        c.Meow();
    }
}
```

Output

```
Animal is eating
Dog is barking
Animal is eating
Cat is meowing
```

Both Dog and Cat inherit the Eat() method from Animal.